Transformer-based Symbolic Music Generation

Symbolic music generation on a small, stylistically consistent corpus
Python / PyTorch / MIDI / MidiTok / music21

422 MIDI arrangements | ~40M parameters | Four representation variants

I developed a symbolic music generation system to explore whether a Transformer trained on a small corpus of stylistically similar music could learn recognisable musical patterns, and whether changing the event representation would improve its outputs. The project grew from building a working generator into an investigation of musical position, harmonic annotation and the limits of automatic evaluation.

GitHub: Archmagos-Lama/text-to-midi-transformer

STARTING POINT

My practical goal was to generate music with some of the character of the source material. I was particularly interested in how a motif can return in a different register or harmonic setting while retaining internal movement: changes in note length, pauses and melodic direction. Matching the instruments or repeating a plausible accompaniment pattern would only address part of that goal.

I chose MIDI because it makes notes, timing and instruments accessible as discrete events. It also allows generated material to be inspected in a piano roll or replayed with different soundfonts. The model generates symbolic events; the final sound still depends on the playback instrument and synthesizer.

SCOPE OF THE INVESTIGATION

The experiments asked three related questions: how closely a small-corpus model could match reference music statistics; how explicit bar indices changed those distributions; and how harmonic-role annotations affected learning and generation. I kept the Transformer architecture broadly fixed while exploring these representation choices.

The system supports generation from a beginning-of-sequence token and continuation from a MIDI prompt. I compared unconditional samples quantitatively and used prompt continuation to explore how the model responded to an existing musical idea.

IMPLEMENTATION

I prepared the data pipeline, implemented the PyTorch model and generation code, ran training and evaluation, and developed the BARPOS and HarmRole variants. MidiTok supplied the underlying REMI tokenizer, while music21 supplied tools for harmonic analysis. I also organised the scripts, saved model checkpoints and inspected generated outputs to identify where the measured results differed from the musical experience.

Building the Pipeline

From MIDI arrangements to an autoregressive event model

DATA AND EVENT REPRESENTATION

The corpus contained 422 MIDI arrangements drawn from a shared musical style. Training and validation material were prepared separately, with pitch-transposition augmentation used to expand the training examples. This exposed the model to the same musical material in different registers and keys.

I used REMI to encode bar boundaries, positions within bars, pitches, velocities, durations, tempo and instrument programs. Beginning- and end-of-sequence markers were added around pieces. The pipeline saved token tensors, tokenizer definitions and metadata so that training and MIDI reconstruction used the same event vocabulary.

TRANSFORMER IMPLEMENTATION AND TRAINING

The decoder-only Transformer used eight layers, eight attention heads and a 640-dimensional embedding, for approximately 40 million parameters. Learned position embeddings represented token positions within a 1,536-token context. The model learned next-token prediction with causal self-attention and cross-entropy loss.

The training configuration used a batch size of 12 and eight gradient-accumulation steps. Validation was evaluated every 500 iterations, and the training scripts saved both recent and best-validation checkpoints. I selected the best-validation checkpoint for sample generation.

HarmRole v1 reached its best validation loss at iteration 2,500; v2 reached its best at iteration 2,000. Further training lowered training loss while validation performance stopped improving, so I stopped the runs and generated samples from the saved best checkpoints.

TESTING BAR-POSITION INFORMATION

Standard REMI uses the same Bar_None marker at every bar boundary. I replaced this marker with BARPOS_k, where k is the bar count modulo 128. This introduces an explicit cyclic bar index without requiring manual phrase or section annotations.

The baseline retained the same 741-entry vocabulary while continuing to use Bar_None. This made the baseline/BARPOS comparison more controlled than the harmonic-role comparisons. During generation, BARPOS tokens were predicted as part of the sequence and mapped back to Bar_None before MIDI decoding.

The hypothesis was that bar indices could help the model learn position-dependent patterns. The index records a bar count, leaving the model to learn any relationship between position and musical content. I assessed its effect by comparing generated feature distributions with the baseline and reference corpus.

The HarmRole Experiment

An annotation idea, an unexpected distribution, and a revised representation

MAKING HARMONY EXPLICIT

After hearing repetitive or poorly organised samples, I explored whether explicit harmonic information could help. The first HarmRole version estimated a chord for each bar and placed a role label immediately after each pitch: root, third, fifth, seventh or non. These labels described the relation between a note and the estimated harmony.

Each role followed its pitch as an auxiliary event in the autoregressive sequence. The model learned to predict pitch-role combinations from context. At generation time it predicted both pitches and labels; the labels stayed in the model context and were removed before MIDI decoding.

KEEPING PREPROCESSING PRACTICAL

Running harmonic analysis separately on every augmented MIDI would repeat much of the same work. I instead analysed the original files with music21, cached the bar-level chord maps, and shifted their pitch classes for each transposed version. Cache persistence also allowed analysis results to be reused after an interrupted run.

For the HarmRole variants, I disabled MidiTok chord extraction while retaining the separately computed role labels. This reduced duplicate preprocessing, but also changed the representation: the vocabulary became 578 entries for v1 and 579 for v2, compared with 741 for baseline and BARPOS. HarmRole also lengthened the sequence, reducing how much musical time fitted within the same token context.

WHAT THE LABELS REVEALED

Counting the training labels showed that **73.8% were non**. Root, third, fifth and seventh together accounted for the remaining 26.2%. The large residual category made the annotation worth investigating before interpreting the generation results.

A single chord per bar provides a coarse description of passages containing passing notes, ornaments and changes of harmony. This grouped many different note functions into the same residual label. The distribution led me to refine the label scheme and examine the assumptions behind the chord and key estimates.

REVISION AND OUTCOME

In v2 I split non into scale and chromatic using an automatically estimated major or minor key. Notes outside the chord but inside that scale received scale; the remaining notes received chromatic. This produced 36.4% scale and 37.4% chromatic labels in the training data.

Version 2 achieved a lower raw validation loss than v1, 1.4375 versus 1.5050, while the musical statistics remained mixed. Since the revised labels also changed the prediction task, I considered the loss alongside the generated outputs. The refinement made the annotation more detailed, but did not produce a consistent gain across the measured musical features.

Results in Context

Measured distributions, generated examples and the remaining musical gap

QUANTITATIVE COMPARISON

I generated 50 samples per variant and compared eight features with reference MIDI: pitch-class entropy, pitch range, note density, average pitch, pitch standard deviation, average duration, rhythmic entropy and polyphony. The evaluator compared feature histograms using Jensen-Shannon distance.

BARPOS matched the reference more closely than baseline on some features, but did not outperform it consistently. HarmRole v1 and v2 also showed improvements on some measurements and regressions on others. The comparison showed how the representation affected the balance of pitch, duration and rhythmic characteristics in the outputs.

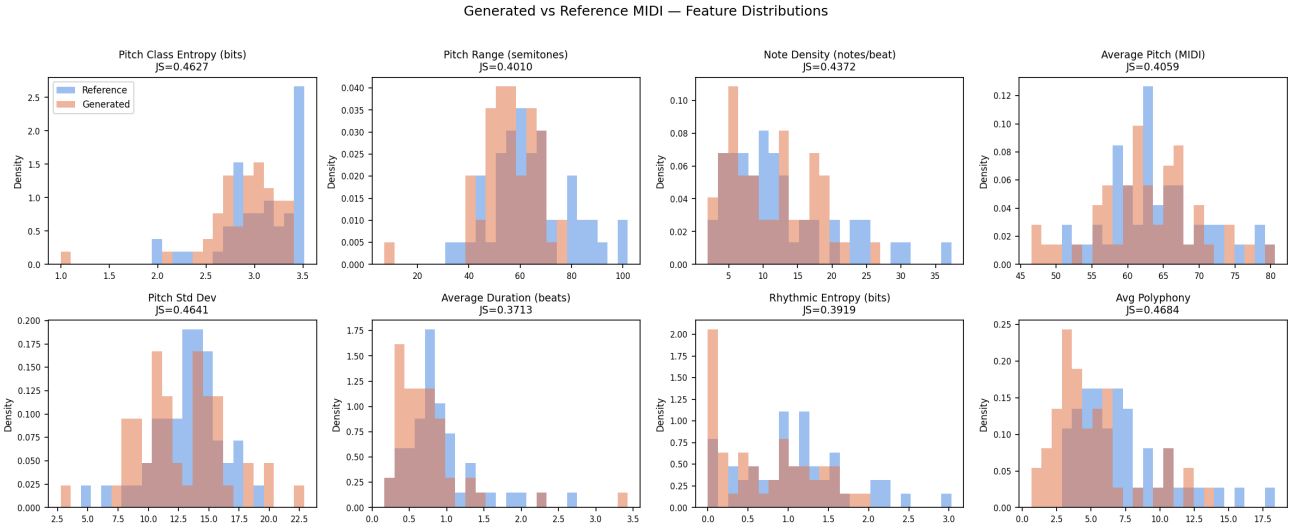


Figure 1. BARPOS-generated MIDI versus reference feature distributions. Blue indicates reference samples and orange indicates generated samples. The original figure labels the histogram comparison as JS; the evaluation code returns Jensen-Shannon distance.

WHAT THE SAMPLES SOUNDED LIKE

Listening exposed a gap between plausible local patterns and coherent musical development. Outputs could imitate a rhythmic texture, instrument setting or chord pattern while repeating similar durations for too long. Motifs often lacked the internal rises, pauses and changes of pace that motivated the project.

I also tested continuation using the first 600 tokens of an existing MIDI as a prompt. This gave the model an established musical idea to work from. The extensions still tended to lose the organisation of the source, showing that providing a convincing opening was not enough to sustain its development.

WHAT THIS PROJECT ESTABLISHES

The completed pipeline connects data preparation, tokenization, training, generation and evaluation. BARPOS changed some feature distributions in a favourable direction; HarmRole led to a detailed annotation revision with mixed generation results. Listening identified the remaining practical problem: sustaining variation and development beyond a plausible local pattern.

The eight statistical features capture aspects of pitch, rhythm and texture. Melodic contour, motif recurrence and phrase organisation remain outside their scope. The main lesson from the project was to examine the representation, the measured distributions and the generated music together when deciding whether a change helped.